

Appendix for “Synthetic Forgetting without Access: A Few-shot Zero-glance Framework for Machine Unlearning”

A Pseudo-code for the GFOES Framework

This section presents pseudo-code for the two main components within the GFOES framework. Algorithm 1 provides a description of the Generative Feedback Network (GFN), which synthesises Optimal Erasure Samples (OES) to support targeted forgetting under zero-glance constraints. Algorithm 2 outlines the two-stage fine-tuning procedure, comprising an erasure phase that removes knowledge related to forgotten classes, followed by a recovery phase that restores model utility on the remaining classes.

Algorithm 1: Generative Feedback Network (GFN)

Require: Initial model $f(x; \theta_0)$, retained data $\mathcal{D}_{rs}$, forgotten class labels Y_f , learning rate η , initial trade-off coefficient λ_0

Ensure: Trained generator $G(z; \phi^*)$

- 1: Initialise generator parameters ϕ and set $\lambda \leftarrow \lambda_0$
- 2: **for** $t = 1$ to T **do**
- 3: Sample noise $z \sim \mathcal{N}(0, 1)$
- 4: Generate synthetic samples $G(z; \phi)$
- 5: **Maximise branch:**
- 6: Compute classification loss on forgotten classes:
- 7: **One-epoch train:**
- 8: **Maximise branch:**
- 9: Compute retained-class loss after one-epoch update:
- 10: **Stabilised Joint Objective:**
- 11: Compute stabilised joint loss:

$$\mathcal{L}_{\max} = \mathcal{L}_{CE}(\mathcal{Y}_f, f(G(z, \phi); \theta_0))$$

$$\theta' \leftarrow \theta_0 - \eta_{GFN} \nabla_{\theta} \mathcal{L}(\mathcal{Y}_f \cup \mathcal{Y}_r, f(G(z, \phi) \cup \mathcal{D}_{rs}; \theta_0))$$

$$L_{\min} = \mathcal{L}_{CE}(Y_r, f(\mathcal{D}_{rs}; \theta'))$$

$$\mathcal{L}_{GFN}^{(t)} = \lambda_t \cdot \frac{1}{\mathcal{L}_{\max}} + (1 - \lambda_t) \cdot \mathcal{L}_{\min}$$

- 12: Update generator parameters:

$$\phi^* \leftarrow \phi - \eta \cdot \nabla_{\phi} \mathcal{L}_{GFN}$$

- 13: Adapt trade-off coefficient:

$$\lambda_{t+1} = \Pi_{(0,1)}(\lambda_t + \eta \cdot \frac{\partial \mathcal{L}_{GFN}^{(t)}}{\partial \lambda})$$

- 14: **end for**

- 15: **return** Trained generator $G(z; \phi^*)$
-

B Convergence Analysis of the GFN

B.1 Instability of the Subtractive Objective

Let $F(\phi)$ and $G(\phi)$ denote the retention and forgetting losses, respectively. The conventional subtractive objective

Algorithm 2: Two-Stage Fine-Tuning

Require: Original model $f(x; \theta_0)$, trained generator $G(z; \phi)$, retained data $\mathcal{D}_{rs}$, learning rates $\eta_{\text{high}}, \eta_{\text{low}}$

Ensure: Unlearned model parameters θ^*

- 1: **Erasure Phase**

- 2: Sample latent code $z \sim \mathcal{N}(0, 1)$

- 3: Generate (OES): $\tilde{x} \leftarrow G(z; \phi^*)$

- 4: Compute erasure loss:

$$\mathcal{L}_{\text{erase}} = \mathcal{L}(\mathcal{Y}_f; f(\tilde{x}; \theta_0)) + \mathcal{L}(\mathcal{D}_{rs}; \theta_0)$$

- 5: Update model:

$$\theta_1 \leftarrow \theta_0 - \eta_{\text{high}} \cdot \nabla_{\theta} \mathcal{L}_{\text{erase}}$$

- 6: **Recovery Phase**

- 7: Compute recovery loss:

$$\mathcal{L}_{\text{rec}} = \mathcal{L}_{CE}(Y_r, f(\mathcal{D}_{rs}; \theta_1))$$

- 8: Update model:

$$\theta^* \leftarrow \theta_1 - \eta_{\text{low}} \cdot \nabla_{\theta} \mathcal{L}_{\text{rec}}$$

- 9: **return** Unlearned model parameters θ^*
-

is defined as:

$$\tilde{\mathcal{L}}(\phi) = F(\phi) - G(\phi)$$

Although this formulation appears intuitive, it presents two main issues:

- **Unboundedness:** As $G(\phi)$ can increase without limit while $F(\phi) \geq 0$, the objective becomes unbounded below, which may result in divergent optimisation paths.
- **Gradient instability:** The competing gradients $\nabla_{\phi} F$ and $\nabla_{\phi} G$ may differ in scale. When $G(\phi)$ dominates, the updates become excessively large, which causes oscillatory behaviour.

B.2 Stabilised Joint Objective

To address these issues, we introduce a stabilised joint objective that incorporates an adaptive balance coefficient $\lambda_t \in (0, 1)$:

$$\mathcal{J}_t(\phi_t, \lambda_t) = \lambda_t \frac{1}{G(\phi_t)} + (1 - \lambda_t) F(\phi_t), \quad \lambda_t \in (0, 1). \quad (15)$$

The parameters are updated through gradient descent with respect to ϕ and projected gradient ascent with respect to λ :

$$\phi_{t+1} = \phi_t - \eta_{\phi} \nabla_{\phi} \mathcal{J}_t(\phi_t, \lambda_t), \quad (16)$$

$$\lambda_{t+1} = \Pi_{[0,1]}(\lambda_t + \eta_{\lambda} \nabla_{\lambda} \mathcal{J}_t(\phi_t, \lambda_t)). \quad (17)$$

B.3 Theoretical Assumptions

We state the following mild assumptions:

- **A1 (Smoothness):** Both $F(\phi)$ and $G(\phi)$ are differentiable and L -smooth. That is,

$$\|\nabla h(\phi) - \nabla h(\phi')\| \leq L\|\phi - \phi'\|, \quad \text{for all } h \in \{F, G\}.$$

This condition holds for cross-entropy losses defined over bounded parameter domains.

- **A2 (Bounded domain):** The feasible set Φ for ϕ is compact, which ensures that $\sup_{\phi \in \Phi} G(\phi) < \infty$.
- **A3 (Positivity):** For all $\phi \in \Phi$, the forgetting loss satisfies $G(\phi) \geq \epsilon > 0$. This property is natural in cross-entropy-based objectives.

B.4 Main Convergence Theorem

The objective defined in Eq. (15) satisfies the following key properties:

Lemma B.1 (Lower bound). *Under Assumptions A2 and A3, the stabilised objective is strictly bounded below:*

$$\mathcal{J}_t(\phi_t, \lambda_t) \geq \frac{\lambda_t}{\sup_{\phi \in \Phi} G(\phi)} > 0.$$

Proof sketch. Both terms in Eq. (15) are non-negative. The reciprocal term $\lambda_t/G(\phi_t)$ is strictly positive due to A3, and the feasible domain ensures boundedness of $G(\phi)$ from above by A2. $\square$

Lemma B.2 (Gradient norm bound). *Suppose that F and G are L -smooth. Then the gradient norm of the objective with respect to ϕ is bounded as:*

$$\|\nabla_{\phi_t} \mathcal{J}_t(\phi_t)\| \leq (1 - \lambda_t)L + \frac{\lambda_t L}{G(\phi_t)^2}.$$

This bound decreases as $G(\phi_t)$ increases, mitigating potential gradient explosion.

Proof sketch. Differentiating Eq. (15) with respect to ϕ_t yields:

$$\nabla_{\phi_t} \mathcal{J}_t(\phi_t) = (1 - \lambda_t)\nabla_{\phi} F(\phi_t) - \lambda_t \cdot \frac{\nabla_{\phi} G(\phi_t)}{G(\phi_t)^2}.$$

Taking norms and applying the triangle inequality:

$$\|\nabla_{\phi_t} \mathcal{J}_t\| \leq (1 - \lambda_t)\|\nabla_{\phi} F\| + \frac{\lambda_t}{G(\phi_t)^2}\|\nabla_{\phi} G\|.$$

If both F and G are L -smooth, then their gradients are Lipschitz and satisfy:

$$\|\nabla_{\phi} F\| \leq L, \quad \|\nabla_{\phi} G\| \leq L.$$

Substituting gives:

$$\|\nabla_{\phi_t} \mathcal{J}_t\| \leq (1 - \lambda_t)L + \frac{\lambda_t L}{G(\phi_t)^2}.$$

As $G(\phi_t)$ increases, the second term decreases, which controls the gradient magnitude and stabilises updates. $\square$

We now rigorously state and prove the convergence theorem for the stabilised GFN training process.

Theorem B.3 (Convergence of GFN Training). *Consider the training of the Generative Feedback Network (GFN) with the stabilised objective defined in Eq. (15), where the parameters (ϕ_t, λ_t) are updated via gradient descent in ϕ and projected gradient ascent in λ . Under Assumptions A1-A3, and with sufficiently small step sizes $\eta_{\phi}, \eta_{\lambda} < 1/L$, the sequence of iterates (ϕ_t, λ_t) converges to a first-order stationary point. Specifically,*

$$\min_{1 \leq t \leq T} \|\nabla_{\phi} \mathcal{J}_t(\phi_t, \lambda_t)\|^2 + \|\nabla_{\lambda} \mathcal{J}_t(\phi_t, \lambda_t)\|^2 \leq \mathcal{O}\left(\frac{1}{T}\right).$$

Proof. The optimisation problem defined by $\mathcal{J}_t(\phi, \lambda)$ is a nonconvex-concave min-max problem. Due to neural network parameterisation and the reciprocal term $1/G(\phi)$, the objective is nonconvex in ϕ . Under Assumptions A1 and A3, where $G(\phi) \geq \epsilon > 0$, the reciprocal term remains continuously differentiable and smooth. Thus, $\mathcal{J}_t$ is a well-behaved, nonconvex smooth function in ϕ .

For fixed ϕ , the objective can be rewritten as

$$\mathcal{J}_t(\phi, \lambda) = \lambda \left(\frac{1}{G(\phi)} - F(\phi) \right) + F(\phi),$$

which is affine in λ , ensuring concavity. Hence, the overall objective is a nonconvex-concave minimax problem.

From Lemma 2, we have the following uniform gradient bound:

$$\|\nabla_{\phi} \mathcal{J}_t(\phi, \lambda)\| \leq (1 - \lambda_t)L + \frac{\lambda_t L}{G(\phi)^2} \leq L + \frac{L}{\epsilon^2},$$

since $G(\phi) \geq \epsilon > 0$. Together with Assumption A2, which ensures that ϕ_t remains in a compact domain, this implies that the iterates (ϕ_t, λ_t) remain bounded and the training process is stable.

To quantify convergence, we define a Lyapunov-like scalar function:

$$V(\phi, \lambda) = \frac{1}{2} (\|\nabla_{\phi} \mathcal{J}_t(\phi, \lambda)\|^2 + \|\nabla_{\lambda} \mathcal{J}_t(\phi, \lambda)\|^2),$$

which measures proximity to a stationary point. If $V(\phi, \lambda) = 0$, then (ϕ, λ) is a first-order stationary point.

We now analyse the optimisation dynamics. From Assumption A1 (smoothness), the descent lemma implies that for $\eta_{\phi} < 1/L$, the gradient descent step in ϕ satisfies:

$$\mathcal{J}_t(\phi_{t+1}, \lambda_t) \leq \mathcal{J}_t(\phi_t, \lambda_t) - \frac{\eta_{\phi}}{2} \|\nabla_{\phi} \mathcal{J}_t(\phi_t, \lambda_t)\|^2.$$

Since $\mathcal{J}_t$ is concave in λ and the domain $[0, 1]$ is convex, the projected gradient ascent step ensures:

$$\mathcal{J}_t(\phi_{t+1}, \lambda_{t+1}) \geq \mathcal{J}_t(\phi_{t+1}, \lambda_t) + \frac{\eta_{\lambda}}{2} \|\nabla_{\lambda} \mathcal{J}_t(\phi_{t+1}, \lambda_t)\|^2.$$

Combining the two inequalities and summing over $t = 1, \dots, T$, we obtain:

$$\begin{aligned} \sum_{t=1}^T (\eta_{\phi} \|\nabla_{\phi} \mathcal{J}_t(\phi_t, \lambda_t)\|^2 + \eta_{\lambda} \|\nabla_{\lambda} \mathcal{J}_t(\phi_t, \lambda_t)\|^2) \\ \leq \mathcal{J}_t(\phi_1, \lambda_T) - \mathcal{J}_t(\phi_{T+1}, \lambda_1) \leq M, \end{aligned}$$

	Learning Rate	Epochs	Batch Size	Weight Decay	Gradient Clipping
AliCNN	4e−4	20	32	1e−4	0.1
ResNet-18	4e−4	100	32	1e−4	0.1
ResNet-50	4e−4	100	32	1e−4	0.1

Table 4: Training protocol for the original models.

	Learning Rate	Epochs	Batch Size	Weight Decay	Gradient Clipping
GFN	4e−3	20	32	1e−4	0.1
Erasure Phase	4e−3	1	32	1e−4	0.1
Recovery Phase	4e−4	1	32	1e−4	0.1

Table 5: Training protocol for the GFOES framework.

for some constant $M > 0$, due to the boundedness of the objective.

Dividing both sides by T and applying the minimum inequality yields:

$$\begin{aligned} \min_{1 \leq t \leq T} (\|\nabla_{\phi} \mathcal{J}_t(\phi_t, \lambda_t)\|^2 + \|\nabla_{\lambda} \mathcal{J}_t(\phi_t, \lambda_t)\|^2) \\ \leq \frac{M}{T \cdot \min(\eta_{\phi}, \eta_{\lambda})} = \mathcal{O}\left(\frac{1}{T}\right). \end{aligned} \quad (18)$$

This result explicitly guarantees that the joint gradient norm converges to zero at a sublinear rate of $\mathcal{O}(1/T)$. As $T \rightarrow \infty$, the iterates (ϕ_t, λ_t) converge to a first-order stationary point. Hence, the training dynamics of the GFN under the stabilised objective are provably convergent. $\square$

C Experimental Settings

This section provides additional details regarding the datasets, model architectures, and training protocols. All experiments are implemented using PyTorch 2.4.1 and conducted on a single NVIDIA RTX 3090 GPU with 24 GB of memory.

C.1 Datasets

We evaluate GFOES on three standard image classification benchmarks:

- **Fashion-MNIST**: Consists of 60,000 grey-scale images (28×28) across 10 categories. We randomly designate 1 or 4 classes as forget classes.
- **CIFAR-10**: Contains 60,000 RGB images (32×32) across 10 object classes. In our setting, 1 or 4 classes are selected as forget classes.
- **CIFAR-100**: Comprises 60,000 RGB images (32×32) from 100 fine-grained categories. We evaluate configurations with 1 or 10 forget classes.

For all datasets, the retained set is down-sampled to 5%, 10%, or 20% of the original per-class instances to ensure class balance. The forget set remains entirely inaccessible during the unlearning process.

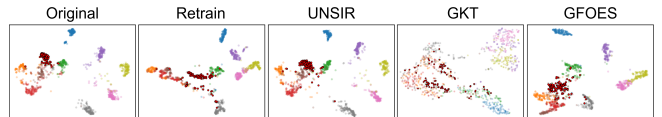


Figure 5: t-SNE visualisation for single-class unlearning on Fashion-MNIST. GFOES disperses feature clusters associated with forgotten classes while preserving well-separated clusters for retained classes.

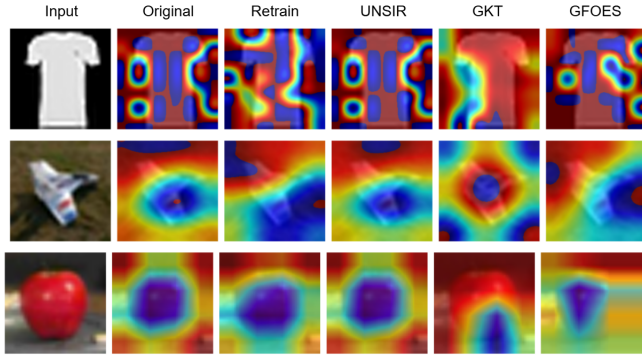
C.2 Model Architectures

We consider three widely used model architectures:

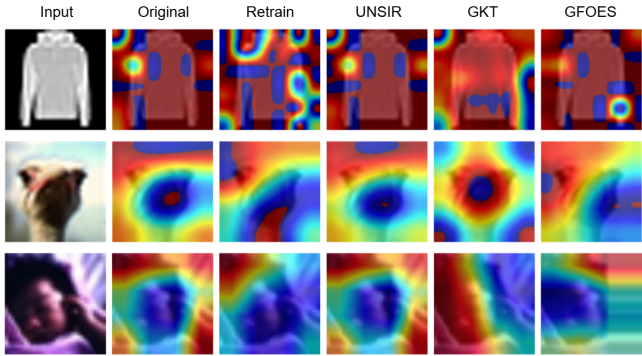
- **AliCNN**: A fully convolutional network comprising 9 layers with ReLU activations. The number of channels increases from 96 (in the first three layers) to 192 (in the next three), followed by class-specific output layers. Downsampling is performed via stride-2 convolutions, and global features are aggregated using adaptive average pooling.
- **ResNet-18**: A lightweight residual network adapted for CIFAR inputs. The initial convolution uses a 3×3 kernel (stride 1, without max pooling). The final fully connected layer is replaced with a dataset-specific linear classifier.
- **ResNet-50**: A deeper residual network initialised with IMAGENET1K_V2 pretrained weights. The initial stage mirrors ResNet-18, and the final layer is modified to output 100 logits for CIFAR-100.

C.3 Training Protocols

We standardise the training setup across all models and methods to enable fair comparison. Table 4 summarises the training configurations used for the original models. For fairness, retraining procedures for all baseline methods follow identical settings. Table 5 details the hyperparameter settings for the GFOES framework. The erasure phase adopts a relatively large learning rate to aggressively remove the target knowledge, followed by a smaller rate in the recovery phase



(a) Single-Class Unlearning



(b) Multi-Class Unlearning

Figure 6: GradCAM heatmaps. (a) Single-class unlearning results on Fashion-MNIST, CIFAR-10, and CIFAR-100; (b) Multi-class unlearning. Warmer colours indicate higher contributions to classification. GFOES eliminates reliance on forgotten-class regions while preserving attention on retained-class areas.

for stabilised fine-tuning. All baseline-specific hyperparameters are adopted directly from their original publications to ensure a fair comparison.

D Additional Experimental Results

D.1 Feature Visualisation (Extended)

In Section 4.3, we present feature visualisation results for single-class unlearning on CIFAR-10. Here, we provide extended results across multiple datasets and unlearning configurations. As shown in Figure 5 and Figure 6, GFOES consistently disrupts the feature structure of forgotten classes while maintaining coherent clusters for retained classes. GradCAM visualisations further confirm that GFOES significantly alters attention regions associated with forgotten classes, reinforcing its ability to remove feature-level knowledge without adversely affecting the representation of retained classes.

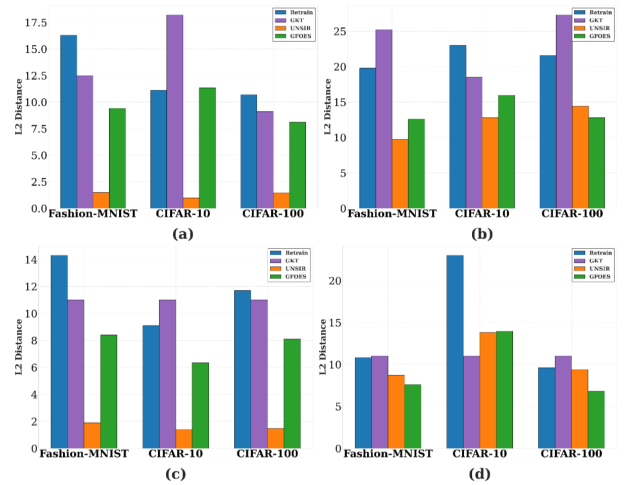


Figure 7: L2 distance between original and unlearned models. (a–b) Single-class unlearning; (c–d) multi-class unlearning. (a, c) Feature extractor; (b, d) final classification layer. GFOES and GKT induce deeper changes at the feature level, whereas UNSIR primarily affects the classification head.

D.2 Multi-class Ablation Study

Table 6 presents ablation results for multi-class unlearning. Consistent with the single-class findings, only the full GFOES configuration achieves *complete forgetting* ($\mathcal{AD}_f = 0$) while maintaining competitive performance on $\mathcal{AD}_r$. Other variants either underperform in forgetting or fail to preserve utility, confirming the necessity of both the OES mechanism and the two-stage learning rate schedule.

D.3 Weight Distance Analysis

We conduct this analysis for both single-class and multi-class unlearning on three datasets. GFOES is compared with GKT and UNSIR, all achieving $\mathcal{AD}_f = 0$. To assess internal changes, we compute the L2 distance between the weights of the unlearned and original models, separating each into the feature extractor and final layer. A larger distance indicates greater deviation and stronger forgetting. As shown in Figure 7, all methods significantly alter the final layer, disrupting classification. However, only GFOES and GKT also modify the feature extractor, indicating deeper removal of class-related knowledge. In contrast, UNSIR preserves this component, maintaining utility but failing to erase latent representations. These results suggest more comprehensive forgetting across both representation and prediction layers.

D.4 Additional Time Efficiency Analysis

Section 4.4 reports the wall-clock time for GFOES and baseline methods, demonstrating the superior efficiency of our approach. Here, we provide additional insights into the time cost breakdown and the design rationale underlying GFOES. As shown in Table 2, the majority of GFOES’s time overhead arises from training the Generative Feedback Network (GFN). At first glance, one might expect that training a generator from scratch would be computationally expensive.

Setting	Fashion		CIFAR-10		CIFAR-100	
	$\mathcal{AD}_f \downarrow$	$\mathcal{AD}_r \uparrow$	$\mathcal{AD}_f \downarrow$	$\mathcal{AD}_r \uparrow$	$\mathcal{AD}_f \downarrow$	$\mathcal{AD}_r \uparrow$
GFOES	0.00	89.12	0.00	87.93	0.00	69.26
OES+D _r +R _l	0.00	58.74	0.00	60.81	0.00	38.29
OES+D _r +R _s	29.82	91.32	24.54	86.51	25.63	71.77
OES+R _{l_s}	0.00	61.28	0.00	55.60	0.00	37.38
OES+R _l	0.00	42.83	0.00	40.34	0.00	30.92
OES+R _s	41.55	78.95	32.63	71.39	25.02	62.91
D _r +R _{l_s}	24.26	76.90	17.73	81.26	18.92	51.29
D _r +R _l	12.39	73.12	7.90	68.66	8.99	52.89
D _r +R _s	42.74	92.97	38.70	88.82	34.99	70.05

Table 6: Ablation results for multi-class unlearning across three datasets. Each setting combines data composition (OES or D_r) and learning rate strategies (R_{l_s}, R_l, or R_s). The best results are shown in **bold**.

However, this is not the case for our design.

The GFN in GFOES fundamentally differs from conventional generative models for realistic image synthesis. Instead of generating visually plausible samples, the GFN optimises a stabilised joint objective to produce pseudo-samples that (i) induce high loss on forgotten classes and (ii) minimise loss on retained classes under the unlearned model. In effect, it learns a structured noise distribution specifically tailored to erase target knowledge while preserving utility. This task-oriented formulation avoids the heavy computational demands typical of full image generation, such as those used in GANs or diffusion models.

As a result, GFOES achieves a favourable trade-off. While GFN training contributes the largest share of computation within GFOES, the total runtime remains an order of magnitude lower than that of retraining or distillation-based zero-shot methods such as GKT. These findings demonstrate that the additional generative step introduces minimal overhead while delivering substantial gains in utility preservation and unlearning quality.